

Mean Average Precision (MAP), Worked Out by Hand

The metric for “did you rank *all* the relevant results well?” — average precision rewards every relevant hit by the precision at its position, and MAP averages that over queries. Built up from precision@ k , computed in full.

What this document does. It computes precision@ k at each relevant position, averages them into Average Precision (AP) for one query, repeats for a second query, and means the two into MAP. It contrasts MAP (all relevants, binary) with MRR (first relevant) and nDCG (graded). Numbers from Appendix A.

Relation to other metrics: MAP is MRR’s multi-answer generalisation and the binary-label sibling of nDCG (#31). It is also exactly the single-query engine behind context precision (#27).

1 What it measures

When a query has *several* relevant documents and you care about all of them being ranked high (not just the first), MRR is too blunt. Average Precision rewards a system for placing *every* relevant document early, and penalises relevant documents that sit behind irrelevant ones. MAP is the mean of AP across queries.

Intuition

Walk down the ranked list and, every time you hit a relevant document, ask “what fraction of everything I’ve seen so far is relevant?” — that is the precision at that point. A relevant document near the top, with little junk above it, scores near 1; one buried under irrelevants scores low. Average those snapshots over all the relevant documents and you get AP: a single number for “how cleanly did the relevants cluster at the top?” MAP just averages AP over your query set.

2 The formula

For one query with R relevant documents, relevance flags $v_k \in \{0, 1\}$:

$$\text{AP} = \frac{1}{R} \sum_{k=1}^K \text{Precision@}k \cdot v_k, \quad \text{Precision@}k = \frac{\sum_{j=1}^k v_j}{k},$$

$$\boxed{\text{MAP} = \frac{1}{Q} \sum_{i=1}^Q \text{AP}_i} \in [0, 1].$$

Only relevant positions ($v_k = 1$) contribute to the sum; dividing by R normalises each query to $[0, 1]$ regardless of how many relevants it has.

3 Worked by hand

Query 1. Ranked relevance $v = (1, 0, 1, 1, 0, 1)$ — relevant at ranks 1, 3, 4, 6, so $R = 4$.

rank k	v_k	rel. so far	Precision@ k (if $v_k=1$)
1	1	1	$1/1 = 1.0000$
2	0	1	—
3	1	2	$2/3 = 0.6667$
4	1	3	$3/4 = 0.7500$
5	0	3	—
6	1	4	$4/6 = 0.6667$

$$AP_1 = \frac{1.0000 + 0.6667 + 0.7500 + 0.6667}{4} = \frac{3.0834}{4} = 0.7708.$$

Query 2. Ranked relevance $v = (0, 1, 0, 0, 1, 0)$ — relevant at ranks 2, 5, so $R = 2$. Precision@2 = $1/2 = 0.5$, Precision@5 = $2/5 = 0.4$:

$$AP_2 = \frac{0.5 + 0.4}{2} = 0.4500.$$

Mean over the two queries.

$$MAP = \frac{0.7708 + 0.4500}{2} = \boxed{0.6104}.$$

Query 1 ranked its relevants well (first one at rank 1); query 2 buried both, so its AP is much lower. MAP, 0.61, is the system's overall ranking quality across the set.

Mini-example • AP rewards relevants-first

In query 1, the relevant document at rank 1 contributed precision 1.0; the one at rank 6 contributed only 0.667 (because two irrelevants sit above it). Had all four relevants been ranked 1–4, every precision would be 1.0 and AP would be a perfect 1.0. AP literally measures how much irrelevant material you let creep above each relevant document — the cleaner the top, the higher the score.

4 MAP vs. MRR vs. nDCG

	relevance	rewards	use when
MRR (#29)	binary	first hit only	one right answer
MAP (#30)	binary	all relevants, ranked	several right answers
nDCG (#31)	graded	all, by grade & position	shades of relevance

MAP is the right metric when each query has multiple correct documents, relevance is yes/no, and you want every correct one near the top — classic retrieval benchmarking. If relevance comes in grades, step up to nDCG; if there is only one right answer, drop down to MRR.

Equation cheat-sheet

$$AP = \frac{1}{R} \sum_k \text{Precision}@k \cdot v_k, \quad MAP = \frac{1}{Q} \sum_i AP_i$$

binary, multi-relevant, rank-aware; one query's AP = context precision (#27)

Appendix A · Reference implementation

```
def average_precision(v):                # v: 1/0 relevance down the ranking
    precs, hits = [], 0
    for k, rel in enumerate(v, start=1):
        if rel:
            hits += 1
            precs.append(hits / k)
    return sum(precs) / len(precs) if precs else 0.0

def mean_ap(queries):
    return sum(average_precision(v) for v in queries) / len(queries)

average_precision([1,0,1,1,0,1])        # 0.7708
average_precision([0,1,0,0,1,0])        # 0.4500
mean_ap([[1,0,1,1,0,1], [0,1,0,0,1,0]]) # 0.6104
```

Internal learning material. Numbers reproducible from the appendix code. v1.0